

115 學年度資訊學科能力競賽臺南一中校內初選 試題本

競賽規則

1. 競賽時間：2026/09/10 13:30 ~ 16:30，共 3 小時。
2. 本次競賽試題共 6 題，每題皆有子任務。
3. 為了愛護地球，本次競賽題本僅提供電子檔，不提供紙本。
4. 每題的分數為該題所有子任務得分數加總；單筆子任務得分數為各筆繳交在該筆得到的最大分數。
5. 本次初選提供記分板。
6. 除通訊題目以外其他題目的輸入皆為標準輸入。
7. 除通訊題目以外其他題目的輸出皆為標準輸出。
8. 所有輸入輸出請嚴格遵守題目要求，多或少的換行及空格皆有可能造成裁判系統判斷為答案錯誤。
9. 每題每次上傳間隔為 120 秒，裁判得視情況調整。
10. 所有試題相關問題請於競賽系統中提問，題目相關公告也會公告於競賽系統，請密切注意。
11. 如有電腦問題，請舉手向監考人員反映。
12. 如有如廁需求，須經過監考人員同意方可離場。
13. 不得攜帶任何參考資料，但競賽系統上的參考資料可自行閱讀。
14. 不得自行攜帶隨身碟，如需備份資料，請將資料儲存於電腦 D 槽。
15. 競賽中請勿交談。請勿做出任何會干擾競賽的行為。
16. 如需使用 C++ 的 `std::cin` 或 `std::cout` 可將以下程式碼插入 `main function` 以及將 `endl` 取代為 `'\n'` 來最佳化輸入輸出速度。唯須注意不可與 `cstdio` 混用。

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

A. 大家好我是今年選訓二階結訓選手

Problem ID: TOI2!

Time Limit: 2s

Memory Limit: 512MiB



「分數高的人贏，分數少的人輸！」是 2026 TOI 選訓營第一階段常聽到的一句話。

選訓營很殘酷，第一階段至第二階段需要從 30 人中選 12 人。

而選拔標準就是通過兩場考試的總分加總比較出來的。

但是大家身為台灣各地的競程高手，內心不免的會有一點點的傲氣。

「懦弱的贏比輸，更可怕！」，這句名言也是由某位學長把 OI 打成 ICPC(不是 0 分就是 100 分的策略) 流傳出來的。

理想很美好，現實很骨感。想要進第二階段還是得乖乖把該拿的分數拿完。

30 人取 12 人，其實可以平均一下，你只要在 30 個人抓 2 個人出來，然後單挑打贏他們兩個，你就可以進二階了。

所以今天身為競程菜雞的 Blame，在第一階段的時候徬徨無助，不知道自己會不會打進第二階段。為了消除心中的不安，Blame 決定寫一個程式來模擬各種賽況。

假設在某場模考中有 N 名選手，每個人都有一個戰力分數。Blame 根據自己那套「單挑理論」，決定將目標設為：「找出這 N 個人當中，分數第 K 低的人，並把他的分數當成自己的假想敵門檻。」

給定 N 名選手的戰力分數 $(A_1, A_2, \dots, A_N)$ ，請幫助徬徨的 Blame 找出由小到大排序後的第 K 個分數，讓他知道自己該拿幾分才能安心！

註：因為有 leolin 那樣一堵很高很大的牆，所以 A_i 高達 10^{18} 是很正常的。

— 輸入 —

第一行包含兩個正整數 N 、 K

第二行包含 N 個整數 $A_1, A_2, \dots, A_N$

— 輸出 —

輸出一個整數，代表陣列由小到大排序後的第 K 個元素。

— 輸入限制 —

- $1 \leq N \leq 2000$
- $1 \leq K \leq N$
- $0 \leq A_i \leq 10^{18}$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	67	$0 \leq A_i \leq 10^9$
3	33	無額外限制

－ 範例輸入 1 －

7 3
9 1 5 1 7 5 3

－ 範例輸出 1 －

3

－ 範例解釋 1 －

將 7 個人的分數按照小到大排序會變成

1, 1, 3, 5, 5, 7, 9

第 3 小的會是 3。

－ 範例輸入 2 －

32 20
247 228 217 214 181 207 203 158 184 175 246 232 211 210 242 132 178 175 114
151 201 146 195 147 131 113 102 100 135 114 78 57

－ 範例輸出 2 －

195

－ 範例輸入 3 －

32 20
200 94 164 172 131 50 78 106 76 102 50 106 33 33 0 109 56 40 100 53 0 50 0 38
50 67 59 50 14 0 3 18

－ 範例輸出 3 －

67

－ Note －

範例輸入 2, 3 是因為第二行太長了，所以才在題本上進行換行，但輸入測資中並不會換行。

B. 好多手錶

Problem ID: Many_Watches

Time Limit: 1s

Memory Limit: 512MiB



Figure 1: 好多手錶

tobiichi3227 最近收到一些手錶，因為這些手錶機芯品質不一，所以有誤差，tobiichi3227 希望將所有手錶的時間調整到相同來比較哪隻手錶誤差最大。

手錶採十二時制，撥超過十二時會從零開始，但這些手錶有個特點，就是時間只能往後調整，不能往前調整。

tobiichi3227 想知道最少總共要調整多少秒才能讓所有手錶的時間都相同。

為了避免調整的時候其他手錶還在跑導致上次時間跑掉，在調整時會讓所有手錶停止運作。

簡單來講 tobiichi3227 收到了 W 隻手錶，且每隻手錶上面的時間會用一整數 t_i 表示。

對於時間為 a 時 b 分 c 秒的手錶來講， t_i 將會是 $a \times 3600 + b \times 60 + c$ 。

而當 t_i 向後調整 $k(k > 0)$ 秒時， t_i 將會變為 $(t_i + k) \bmod 43200$

— 輸入 —

第一行有一個整數 W ，代表 tobiichi3227 收到 W 隻手錶

第二行有 W 個整數 t_i ($1 \leq i \leq W$)，代表每支手錶現在的時間

— 輸出 —

總共最少要撥動多少秒才會讓時間都相同

— 輸入限制 —

- $1 \leq W \leq 2 \times 10^5$
- $0 \leq t_i < 43200$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	2	所有 t_i 相同
3	5	$W = 2$
4	12	$W \leq 8$
5	18	$W \leq 2000$
6	26	$W \leq 20000$
7	37	無額外限制

— 範例輸入 1 —

1
3227

— 範例輸出 1 —

0

— 範例解釋 1 —

一個手錶，根本不用調整，直接輸出 0

－ 範例輸入 2 －

3
43199 2 5

－ 範例輸出 2 －

9

－ 範例解釋 2 －

同步到 5，總成本為 $6 + 3 + 0 = 9$

- $43200 - 43199 + 5 = 6$
- $5 - 2 = 3$
- $5 - 5 = 0$

－ 範例輸入 3 －

4
0 100 200 40000

－ 範例輸出 3 －

3700

－ 範例解釋 3 －

同步到 200，總成本為 $200 + 100 + 0 + 3400 = 3700$

- $200 - 0 = 200$
- $200 - 100 = 100$
- $200 - 200 = 0$
- $43200 - 40000 + 200 = 3400$

C. 撞球

Problem ID: Pool_Ball

Time Limit: 1s

Memory Limit: 512MiB

傳奇斯諾克巨星「火箭」奧沙利文 (Ronnie O' Sullivan) 以快如閃電的擊球速度與鬼斧神工的母球路徑聞名於世。今天，他的老對手兼南區賽佳作 aka 一中地頭蛇 charhao 在球具店設計了一張特殊的撞球檯。

這張撞球檯可以看作一個有 N 列、 M 欄的方格網格，周圍由高彈性的外框包圍。charhao 在球檯的某些格子上擺放了特殊的可旋轉雙面斜角邊框：

- 空格 .：母球可自由穿過，維持原方向直線前進。
- 斜線 / 或 \：母球碰到時會進行 90 度完全彈性反射。

球檯外框圍繞著 $2(N + M)$ 個編號為 1 到 $2(N + M)$ 的出口/入口（順時針編號）：

1. 頂邊 (Top)：從左到右編號為 $1 \sim M$ （由上往下射入第 1 到 M 欄）。
2. 右邊 (Right)：從上到下編號為 $M + 1 \sim M + N$ （由右往左射入第 1 到 N 列）。
3. 底邊 (Bottom)：從右到左編號為 $M + N + 1 \sim 2M + N$ （由下往上射入第 M 到 1 欄）。
4. 左邊 (Left)：從下到上編號為 $2M + N + 1 \sim 2M + 2N$ （由左往右射入第 N 到 1 列）。

charhao 準備了 Q 個問題想考考奧沙利文，每次詢問會指定一個入口 a 。奧沙利文號稱只要 0.1 秒就能精確算出一顆母球從入口 a 垂直射入後，經過無數次反射最終會從哪一個出口射出。

但這次 charhao 設計的障礙太複雜了，請你寫一個程式幫助 charhao 驗證奧沙利文的答案是否正確！

— 輸入 —

第一行包含三個正整數 N, M, Q ，分別代表球檯的列數、欄數以及詢問次數。

接下來有 N 行，每行包含一個長度為 M 的字元字串，字元只會是 $\cdot$ 、 $/$ 或 $\backslash$ ，代表球檯內部的佈局。

接下來有 Q 行，每行包含一個整數 a ，代表母球射入的外框入口編號。

— 輸出 —

對於每次詢問，請輸出一行包含一個整數，代表母球最後離開球檯時的出口編號。

假如射進去的母球最終無法離開球檯，則輸出 -1 。

— 輸入限制 —

- $1 \leq N, M \leq 2000$
- $1 \leq Q \leq 2 \cdot 10^5$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	3	$NM = 1$
3	5	$1 \leq Q \leq 10$
4	5	$N = 1$
5	7	$N = 2$
6	7	只會有 $\cdot$ 與 $/$ 或 $\backslash$
7	7	所有詢問都從上方進入
8	8	每一列最多有一個 $\backslash$ 或 $/$
9	7	$1 \leq N, M \leq 10, 1 \leq Q \leq 10$
10	8	$1 \leq N, M \leq 1000, 1 \leq Q \leq 100$ ，輸入的字元僅包含 $\cdot$
11	18	$1 \leq N, M \leq 200, 1 \leq Q \leq 1000$
12	25	無額外限制

— 範例輸入 1 —

3 4 8
 ./..
 ..\
 \../
 1
 4
 5
 7
 8
 11
 13
 14

— 範例輸出 1 —

4
 1
 10
 8
 7
 12
 9
 2

— 範例解釋 1 —

以入口 5 為例，球從右側進入第 1 列，向左移動到第 1 列第 2 欄的 / 後轉向下方，最後由底邊的出口 10 離開。路徑如下，其中箭頭表示球的移動方向：

八次詢問所對應的入口與出口依序為

$1 \rightarrow 4, \quad 4 \rightarrow 1, \quad 5 \rightarrow 10, \quad 7 \rightarrow 8, \quad 8 \rightarrow 7, \quad 11 \rightarrow 12, \quad 13 \rightarrow 9, \quad 14 \rightarrow 2.$

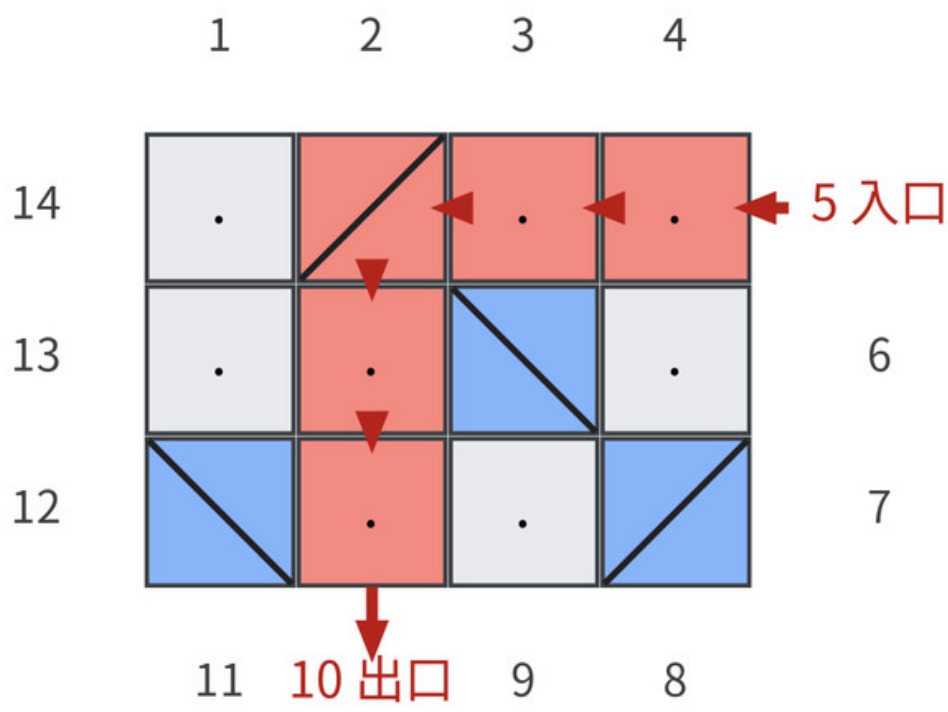


Figure 1: 範例一中，球由入口 5 移動至出口 10 的路徑

— 範例輸入 2 —

```
1 1 4
/
1
2
3
4
```

— 範例輸出 2 —

```
4
3
2
1
```

— 範例解釋 2 —

唯一的格子中放置了 /。從頂邊入口 1 射入的球會向左反射，從出口 4 離開；從右邊入口 2 射入的球則會向下反射，從出口 3 離開。

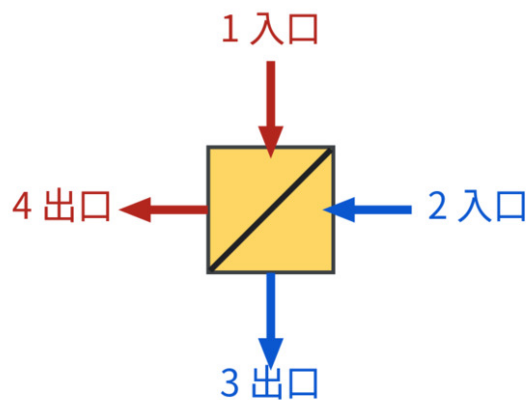


Figure 2: 範例二的兩條反射路徑；紅色為 $1 \rightarrow 4$ ，藍色為 $2 \rightarrow 3$

路徑可以反向行進，因此另外兩次詢問的答案分別為 $3 \rightarrow 2$ 與 $4 \rightarrow 1$ 。

— 範例輸入 3 —

```
2 3 5
.\.
//.
8
10
2
4
5
```

— 範例輸出 3 —

```
10
8
4
2
7
```

— 範例解釋 3 —

以入口 8 為例，球由底邊進入第 1 欄，依序在 (2, 1) 的 /、(2, 2) 的 / 與 (1, 2) 的 \ 反射，最後由出口 10 離開：

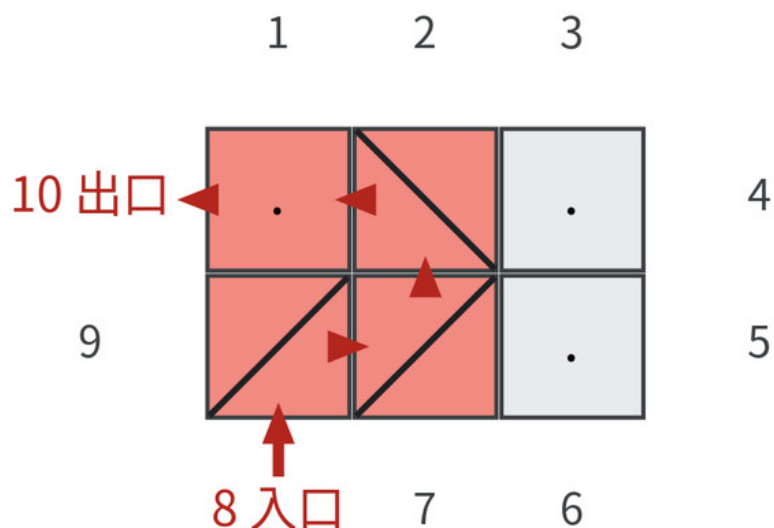


Figure 3: 範例三中，球由入口 8 經過三次反射後抵達出口 10

反向從入口 10 進入時會沿同一條路徑從出口 8 離開。另外，入口 2 與出口 4 互相連通，入口 5 則與出口 7 互相連通，因此五次詢問的答案依序為 10、8、4、2、7。

D. 1145141

Problem ID: 1145141

Time Limit: 1s

Memory Limit: 128MiB

給定一個正整數 n ，求 $n!$ 的正因數個數。

— 輸入 —

第一行包含一個正整數 n 。

— 輸出 —

答案可能很大，請輸出答案除以 1145141 的餘數。

— 輸入限制 —

- $1 \leq n \leq 10^9$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	5	$1 \leq n \leq 5$
3	11	$1 \leq n \leq 15$
4	45	$1 \leq n \leq 10^3$
5	26	$1 \leq n \leq 10^6$
6	6	$1 \leq n \leq 10^8$
7	7	無額外限制

— 範例輸入 —

3

— 範例輸出 —

4

— 範例解釋 —

對於 $3! = 6$ 來講，有著正因數

1, 2, 3, 6 四個正因數

所以答案為 4

E. 分蛋糕 4

Problem ID: Cake_4

Time Limit: 1s

Memory Limit: 512MiB

本題為互動題，需要實作兩個檔案。

南宮柳棲今天邀請了資訊社的大家參加他的派對。派對中有一個超大超大的蛋糕，從左到右編號為第 0 段至第 $n - 1$ 段。

為了追求公平，蛋糕必須被徹底切分，直到所有蛋糕都成為單獨的一小塊。對於任何一段連續的蛋糕區間 $[l, r]$ ，切蛋糕的人可以選擇一個位置 k ($l \leq k < r$) 切一刀，將其分成 $[l, k]$ 與 $[k + 1, r]$ 兩塊，接著再分別將左右兩塊徹底切分。

對每個區間 $[l, r]$ ，都有一個事先決定的非負整數 $w_{l,r}$ 。每當對區間 $[l, r]$ 切下第一刀時，柳棲想要跳起來大喊「6767」的衝動值會增加 $w_{l,r}$ 。單獨一小塊不需要再切，因此 $w_{i,i} = 0$ 。

舉例來說，如果選擇在位置 k 切開 $[l, r]$ ，這次完整切分所產生的總衝動值為 $w_{l,r} + \text{cost}(l, k) + \text{cost}(k + 1, r)$

其中 $\text{cost}(x, y)$ 代表將區間 $[x, y]$ 徹底切分所需的最小總衝動值。

這份費用表 w 雖然是事先決定的，但柳棲知道它完全反映了蛋糕的物理尺寸與視覺張力，並保證滿足兩個極為自然的規律：

- **大就是好**：蛋糕區間越長，視覺震撼越強。對於任何一段連續的蛋糕，如果只取其中一部分連續的子區間來切，該子區間產生的衝動值絕對不會超過原本整段大區間的衝動值。
- **越長越好**：蛋糕的視覺衝擊具有邊際遞增的效應。想像有兩段右邊界對齊、但左邊界不同的蛋糕區間（即一長一短）。如果將這兩段蛋糕的右邊界同時向右延伸一段相同的長度，那麼原本就比較長（左邊界較遠）的那段蛋糕，它所產生的「衝動值增加量」，一定大於等於較短蛋糕的衝動值增加量。

柳棲有著預知未來的能力，他知道完整的費用表 w 。但是，在派對上實際負責切蛋糕的巴漆並不知道任何 $w_{l,r}$ 。

柳棲只能在派對開始前，傳送一段長度不超過 B 個位元的二進位編碼給巴漆。派對開始後，現場會出現 Q 個詢問。每次詢問會給定一個區間 $[l, r]$ ，巴漆必須回答：**如果只拿出區間 $[l, r]$ 的蛋糕進行徹底切分，為了讓總衝動值最小，第一刀需要切在哪個位置 k ？**如果在多個位置切第一刀都能得到最小總衝動值，回答其中任意一個即可。

— 實作說明 —

本題是 **Two Step** 題型。你需要上傳兩個檔案。

柳棲

第一個檔案為 `sixseven.cpp`，你必須在該檔案內實作 `encode` 這個函式：

```
std::string encode(int n, const std::array<std::array<long long, 2000>, 2
```

- n 是蛋糕的總段數。
- w 是一個 2000×2000 的二維陣列。
- 對所有 $0 \leq l \leq r < n$ ， $w[l][r]$ 等於 $w_{l,r}$ 。
- 當 $l > r$ 時， $w[l][r]$ 的內容沒有意義，請勿使用。
- 對於所有 $n \leq i, j < 2000$ 的 $w[i][j]$ ，其內容沒有意義，請勿使用。
- 你必須回傳一個只包含字元 0 與 1、長度不超過 B 的字串。

巴漆

第二個檔案為 `eightseven.cpp`，你必須在該檔案內實作 `decode` 與 `query` 兩個函式：

```
void decode(int n, const std::string& S);
```

- n 是蛋糕的總段數。
- S 是第一個程式中 `encode` 回傳的二進位字串。
- 第二個程式不會取得費用表 w 。

```
int query(int l, int r);
```

- l, r 代表本次詢問的蛋糕區間 $[l, r]$ 。
- 你必須回傳一個最佳的第一刀切點 k ，且 $l \leq k < r$ 。
- 如果有多個最佳切點，回傳其中任意一個即可。

— 執行細節 —

- 你上傳的兩個檔案會與評測端準備的 `stub.cpp` 一同編譯成一個執行檔 `Cake_4`，你可以在兩個檔案內宣告任何全域變數或內部函數，但請將這些函數宣告在匿名命名空間內以避免變數撞名，詳細請參考範例程式碼。
- 評測端會額外準備 `manager.cpp` 來操作兩個行程之間的互動。具體來說，評測端在編譯出兩份執行檔 `manager` 和 `Cake_4` 後，兩份執行檔將會同時運行，並用以下方式測試：
 1. 評測端建立兩個不同的行程 `sixseven` 與 `eightseven` (`sixseven` 與 `eightseven` 都是執行 `Cake_4` 得到) 以及 `manager` 行程
 2. `manager` 從輸入檔讀入輸入
 3. `manager` 將所需資料透過 `pipe` 傳給程式 `sixseven`，並呼叫 `encode()`
 4. `sixseven` 在收到 `encode()` 的回傳值後，將回傳值透過 `pipe` 傳給 `manager`
 5. `manager` 將所需資料透過 `pipe` 傳給程式 `eightseven`，並呼叫 `decode()`
 6. 接著 `manager` 將會執行 Q 次的詢問，每次呼叫 `query()`
 7. 每次詢問的時候，`eightseven` 收到 `query()` 的回傳值，將回傳值透過 `pipe` 回傳給 `manager` 並判斷結果
- 評分時評測端會建立兩個不同且隔離的行程 (Process)，這代表兩分程式之間的全域變數或內部函數是不能共用的。你也不能使用任何方法來和任何外部檔案互動，任何對 `stdout`、`stderr` 的輸出會被忽略。
- `manager.cpp` 跟 `stub.cpp` 的所花時間微小至可忽略不計。

— 測資限制 —

- $2 \leq n \leq 2000$
- $0 \leq w_{l,r} \leq 3 \times 10^{15}$
- $w_{i,i} = 0$
- 費用表 w 滿足題目開頭所述的兩項性質
- $1 \leq Q \leq 10^5$
- $B = 4 \times 10^6$
- 每次詢問皆滿足 $0 \leq l < r < n$

— 互動範例 —

考慮附件中第一筆範例測試資料的情境

$$n = 4, Q = 3$$

$$w = \begin{pmatrix} 0 & 1 & 3 & 19 \\ & 0 & 2 & 18 \\ & & 0 & 16 \\ & & & 0 \end{pmatrix}.$$

一個可能被評為 Accepted 的互動例子顯示如下：

評分程式端	參賽者端
呼叫 <code>encode(4, W)</code>	
	回傳 S
呼叫 <code>decode(4, S)</code>	
呼叫 <code>query(0, 3)</code>	
	回傳 2
呼叫 <code>query(0, 1)</code>	
	回傳 0
呼叫 <code>query(1, 3)</code>	
	回傳 2

- W 的值為 $\{\{0, 1, 3, 19\}, \{87, 0, 2, 18\}, \{87, 87, 0, 16\}, \{87, 87, 87, 0\}\}$
- S 的值為 “100010101000010101010101111010100010” (我隨便亂打的)

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	27	$2 \leq n \leq 50, 1 \leq Q \leq 1000$
3	18	$2 \leq n \leq 200$
4	41	$2 \leq n \leq 800$
5	14	無額外限制

— 範例評分程式 —

為了測試方便，範例評分程式不完全按照執行細節段落內所講的執行，而是會將你的程式編譯成單一執行檔執行後，直接在該次執行依序呼叫 `sixseven` 和 `eightseven` 兩支程式。請注意，在範例評分程式執行的過程中，`sixseven` 與 `eightseven` 將可以共享不在匿名空間內的全域變數或內部函數，**但這在正式評分程式是不允許的**，因此還請特別注意別誤共享了資訊。

請使用 `compile_cpp.sh` 或 `compile_cpp.bat` 來編譯你寫好的兩支程式。並使用 `run_cpp.sh` 或 `run_cpp.bat` 執行範例評分程式，它將採用以下格式輸入：

```

 $n$     $Q$ 
 $w_{0,0}, w_{0,1}, \dots, w_{0,n-1}$ 
 $w_{1,1}, w_{1,2}, \dots, w_{1,n-1}$ 
 $\vdots$ 
 $w_{n-1,n-1}$ 
 $l_1, r_1$ 
 $l_2, r_2$ 
 $\vdots$ 
 $l_Q, r_Q$ 

```

範例評分程式首先呼叫 `encode(n, w)`。接著，若範例評分程式偵測到 `encode()` 的回傳值有任何不合法，此程式將輸出

```
wrong answer: msg
```

後並終止程式執行，而 `msg` 為下列其中之一錯誤訊息。

- `Length limit exceeded` : `encode()` 的回傳字串長度超過 B 個字元。
- `Invalid returned character` : `encode()` 的回傳字串不是一個 01 字串。

否則，範例評分程式將會輸出一行

```
S
```

接著呼叫 `decode(n, S)`，其中的 S 將會是呼叫 `encode()` 所拿到的回傳字串。

接著範例評分程式會依序執行 Q 次詢問，每次呼叫 `query(l, r)`，並以以下格式印在標準輸出中：

```
i : query(l, r) k
```

- l, r 在第 i 次詢問中為 l_i, r_i
- k 為 `query(l, r)` 所得到的回傳值

F. 種馬拉車

Problem ID: Seed_Manacher

Time Limit: 1s

Memory Limit: 512MiB



曼波，曼波。哈基米曼波～

賽馬娘待兼詩歌劇 (Matikanetannhauser) 是一位性格非常認真且充滿正能量的馬娘。

在中京賽馬場草地 1700 米新馬戰中，雖然詩歌劇展現了出色實力並獲得了第二人氣，但最終的賽果卻讓她不甚滿意。為了能在下一場比賽大放異彩，她決定在賽後進行嚴格的自我檢討！

詩歌劇仔細研究了賽道後發現，中京草地賽場可以抽象化為一個長度為 $2n$ 的陣列 A 。主辦方為了賽道的生態平衡，在場地上均勻地種植了 n 種作物，編號分別為 $0 \sim n - 1$ ，且每種作物恰好出現 2 次。

詩歌劇發現，當她跑過某個連續區間 $[l, r]$ ($1 \leq l \leq r \leq 2n$) 時：

- 令子陣列 $B = [A_l, A_{l+1}, \dots, A_r]$ 。
- 若 B 為一個迴文序列（即 B 與其倒序 B' 完全相同），則詩歌劇在此區間可以獲得 $\max(A_l, A_{l+1}, \dots, A_r)$ 的「前進 Mambo 魔力」。
- 若 B 不是迴文序列，則獲得的魔力為 0。

詩歌劇迫切想知道，到底是自己當天發揮失常，還是場地限制了她的 Mambo 魔力？請身為臺南賽區種馬選手的你，幫詩歌劇計算出陣列 A 中所有可能子區間 $[l, r]$ 的前進 Mambo 魔力總和吧！

－ 輸入 －

第一行輸入一正整數 n

第二行輸入 $2n$ 個非負整數 $A_1, A_2, \dots, A_{2n}$

－ 輸出 －

輸出一整數，代表著答案。

－ 輸入限制 －

- $1 \leq n \leq 10^6$
- $0 \leq A_i \leq n - 1, (1 \leq i \leq 2n)$
- $\sum_{i=1}^{2n} [A_i == j] = 2, (0 \leq j \leq n - 1)$

－ 子任務 －

編號	分數	額外限制
1	0	範例輸入輸出
2	14	$1 \leq n \leq 100$
3	25	$1 \leq n \leq 2000$
4	61	無額外限制

－ 範例輸入 1 －

2
0 1 0 1

－ 範例輸出 1 －

4

－ 範例解釋 1 －

對於陣列 A 來講，總共有 6 個連續區間為迴文序列：

$[1, 1], [2, 2], [3, 3], [4, 4], [1, 3], [2, 4]$ ，分別對答案貢獻 0, 1, 0, 1, 1, 1

總答案為 $0 + 1 + 0 + 1 + 1 + 1 = 4$

－ 範例輸入 2 －

3
0 1 2 2 1 0

－ 範例輸出 2 －

12

115 學年度資訊學科能力競賽臺南一中校內初選 試題本

附錄

參考公式:

- 和的平方公式： $(a + b)^2 = a^2 + 2ab + b^2$
- 差的平方公式： $(a - b)^2 = a^2 - 2ab + b^2$
- 平方差公式： $a^2 - b^2 = (a + b)(a - b)$
- 若直角三角形兩股長為 a 、 b ，斜邊長為 c ，則 $c^2 = a^2 + b^2$
- 若圓的半徑為 r ，圓周率為 π ，則圓面積 $= \pi r^2$ ，圓周長 $= 2\pi r$
- 凸 n 邊形的內角和為 $(n - 2) \times 180^\circ$ （其中 $n \geq 3$ ）
- 若一個等差數列的首項為 a_1 ，公差為 d ，第 n 項為 a_n ，前 n 項和為 S_n ，則：
$$a_n = a_1 + (n - 1)d, S_n = \frac{n(a_1 + a_n)}{2}$$
- 若一個等比數列的首項為 a_1 ，公比為 r ，第 n 項為 a_n ，則 $a_n = a_1 r^{n-1}$
- 一元二次方程式 $ax^2 + bx + c = 0$ 的解為 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

祝各位都能破台，GL & HF